Chapter 4

Defining functions

Understanding scope

- Previous examples have used built-in functions of the Python programming language, such as the **print()** function.
- However, most Python programs contain a number of custom functions that can be called as required when the program runs.

NOTE: Function statements must be indented from the definition line by the same amount so the Python interpreter can recognize the block.

- A custom function is created using the **def (definition)** keyword followed by a name of your choice and ( ) parentheses.
- This line must end with a **:** colon character, then the statements to be executed whenever the function gets called must appear on lines below and indented.
- Syntax of a function definition, therefore, looks like this:

```
def function-name ( ) :
    statements-to-be-executed
    statements-to-be-executed
```

- Once the function statements have been executed, program flow resumes at the point directly following the function call.
- To create custom functions it is necessary to understand the accessibility (“scope”) of Variables in a program:
  - Variables created outside functions can be referenced by statements inside functions –they have “**global**” scope
  - Variables created inside functions cannot be referenced from outside the function in which they have been created– these have “**local**” scope

- If you want to coerce a local variable to make it accessible elsewhere it must first be declared with the Python **global** keyword followed by its name only.
- It may subsequently be assigned a value that can be referenced from anywhere in the program.
- Where a global variable and a local variable have the same name the function will use the local version.

To demonstrate local and global variables

```
global_var = 1
def my_vars() :
    print( 'Global Variable:' , global_var )
    local_var = 2
    print( 'Local variable:' , local_var )
    global inner_var
    inner_var = 3
my_vars()
print( 'Coerced Global:' , inner_var )
```

Supplying arguments

- When defining a custom function in Python programming you may, optionally, specify an “argument” name between the function’s parentheses.
- A value can then be passed to that argument by specifying the value in the parentheses of the call to the function.
- The function can now use that passed in value during its execution by referencing it via the argument name.

```
def echo( user ) :
    print( ‘User:’ , user )
```

- Multiple arguments (a.k.a. “parameters”) can be specified in the function definition by including a comma-separated list of argument names within the function parentheses:

```
def echo( user , lang , sys ) :
    print( User:’ , user , ‘Language:’ , lang , ‘Platform:’ , sys )
```

- When calling a function whose definition specifies arguments, the call must include the same number of data values as arguments. For example, to call this example with multiple arguments:

```
echo( ‘Mike’ , ‘Python’ , ‘Windows’ )
```

- The passed values must appear in the same order as the arguments list unless the caller also specifies the argument names like this:

```
echo( lang = ‘Python’ , user = ‘Mike’ , sys = ‘Windows’ )
```

```
def echo1( user , lang , sys ) :
    print( 'User:', user, 'Language:', lang, 'Platform:', sys )

echo1( 'Mike' , 'Python' , 'Windows' )
echo1( lang = 'Python' , sys = 'Mac OS' , user = 'Anne' )

def mirror( user = 'Carole' , lang = 'Python' ) :
    print( '\nUser:' , user , 'Language:' , lang )

mirror()
mirror( lang = 'Java' )
mirror( user = 'Tony' )
mirror( 'Susan' , 'C++' )
```

Returning values

- Custom functions can also return a value to their caller by using the Python return keyword to specify a value to be returned.
- For example, to return to the caller the total of adding two specified argument values like this:

```
def sum( a , b ) :
    return a + b
```

- The returned result may be assigned to a variable by the caller for subsequent use by the program like this:

```
total = sum( 8 , 4 )
print( 'Eight Plus Four Is:' , total )
```

- Or the returned result may be used directly "in-line" like this:

```
print( 'Eight Plus Four Is:' , sum( 8 , 4 ) )
```

- Typically, a **return** statement will appear at the very end of a function block to return the final result of executing all statements contained in that function.
- A return statement may, however, appear earlier in the function block to halt execution of all subsequent statements in that block.
- This immediately resumes execution of the program at the caller. Optionally, the **return** statement may specify a value to be returned to the caller or the value may be omitted.

- Where no value is specified, a default value of **None** is assumed. Typically, this is used to halt execution of the function statements after a conditional test is found to be **False.**

Example:

```
def sum( a , b ) :
    if a < 5 :
        return
    return a + b
```

Default arguments

- A default argument is an argument that assumes a default value if a value is not provided in the function call for that argument.
- The following example gives an idea on default arguments, it prints default age if it is not passed

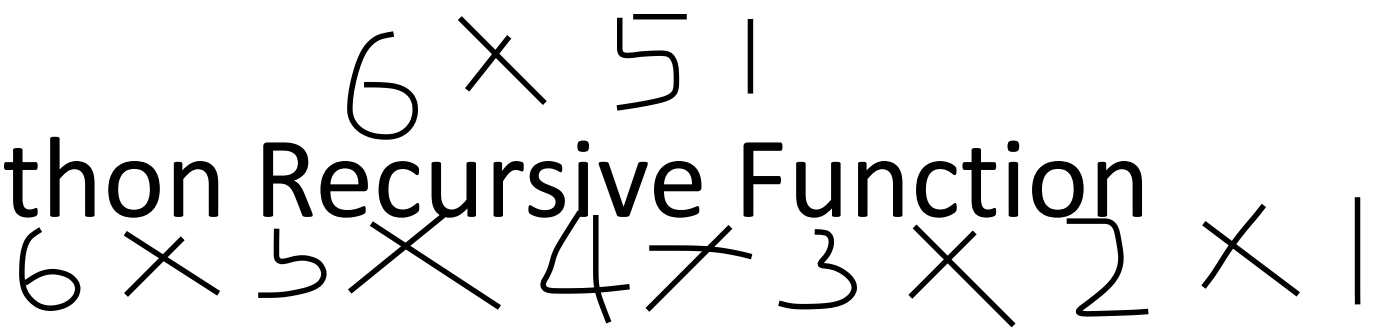
Default arguments

```
# Function definition is here
def printinfo( name, age = 35 ):
"This prints a passed info into this function"
print "Name: ", name
print "Age ", age
return;
# Now you can call printinfo function
printinfo( age=50, name="miki" )
printinfo( name="miki" )
```

```
Name: miki
Age 50
Name: miki
Age 35
```

What is recursion?

- Recursion is the process of defining something in terms of itself.
- A physical world example would be to place two parallel mirrors facing each other.
- Any object in between them would be reflected recursively.



Python Recursive Function

- We know that in Python, a function can call other functions. It is even possible for the function to call itself. These type of construct are termed as recursive functions.
- Following is an example of recursive function to find the factorial of an integer.
- Factorial of a number is the product of all the integers from 1 to that number. For example, the factorial of 6 (denoted as 6!) is 1*2*3*4*5*6 = 720.

```python
# An example of a recursive function to
# find the factorial of a number

def calc_factorial(x):
    """This is a recursive function
    to find the factorial of an integer"""

    if x == 1:
        return 1
    else:
        return (x * calc_factorial(x-1))

num = 4
print("The factorial of", num, "is", calc_factorial(num))
```

if x==1 or x==0:

Python Recursive Function

- In the above example, calc_factorial() is a recursive functions as it calls itself.
- When we call this function with a positive integer, it will recursively call itself by decreasing the number.
- Each function call multiples the number with the factorial of number 1 until the number is equal to one. This recursive call can be explained in the following steps.

Python Recursive Function

```
calc_factorial(4)               # 1st call with 4
4 * calc_factorial(3)           # 2nd call with 3
4 * 3 * calc_factorial(2)       # 3rd call with 2
4 * 3 * 2 * calc_factorial(1)   # 4th call with 1
4 * 3 * 2 * 1                   # return from 4th call as number=1
4 * 3 * 2                       # return from 3rd call
4 * 6                           # return from 2nd call
24                              # return from 1st call
```

Our recursion ends when the number reduces to 1.
This is called the base condition.
Every recursive function must have a base condition that stops the recursion or else the function calls itself infinitely.

Challenges

- Calculate the sum of the first n natural numbers using recursion.
- Generate Fibonacci sequence using recursion
- Write a Python function to calculate the factorial of a number (Iterative).
- Write a menu driven program to add, subtract, multiply and divide two integers.
- Write a program that takes a list of numbers (for example, a = [5, 10, 15, 20, 25]) and makes a new list of only the first and last elements of the given list. For practice, write this code inside a function.

Additional Material

Using callbacks

- Python also allows an anonymous un-named function to be created using the **lambda** keyword.
- An anonymous function may only contain a single expression which must always return a value.
- The lambda keyword, therefore, offers the programmer an alternative syntax for the creation of a function.

- For example:

```
def square( x ) :
    return x ** 2
```

can alternatively be written more succinctly as…

```
square = lambda x : x ** 2
```

- In either case, the call **square(5)** returns the result 25 by passing in an integer argument to the function.

Note: the **lambda** keyword is followed by an argument without parentheses and the specified expression does not require the return keyword as all functions created with lambda must implicitly return a value.

Adding placeholders

- The Python **pass** keyword is useful when writing program code as a temporary placeholder that can be inserted into the code at places where further code needs to be added later.
- The **pass** keyword is inserted where a statement is required syntactically but it merely performs a “null” operation – when it is executed nothing happens and no code needs to be executed.
- This allows an incomplete program to be executed for testing by simulating correct syntax so the interpreter does not report errors.

```
bool = True
if bool :
    print( ‘Python In Easy Steps’ )
else :
    # Statements to be inserted here.
```

NOTE: In a loop the **continue** keyword continues on the next iteration, whereas the **pass** keyword passes on to the next statement of the same iteration.

```
title = '\nPython In Easy Steps\n'
for char in title : print( char , end = ' ' )

for char in title :
    if char == 'y' :
        print( '*' , end = ' ' )
        continue
    print( char , end = ' ' )

for char in title :
    if char == 'y' :
        print( '*' , end = ' ' )
        pass
    print( char , end = ' ' )
```

Producing generators

- A Python generator, on the other hand, is a special function that returns a “generator object” to the caller rather than a data value.
- This, effectively, retains the state of the function when it was last called so it will continue from that point when next called
- Generator functions are produced by definition just like regular functions but contain a “yield” statement. This begins with the Python yield keyword and specifies the generator object to be returned to the caller.
- When the yield statement gets executed, the state of the generator object is frozen and the current value in its “expression list” is retained.

Handling exceptions

- Sections of a Python script in which it is possible to anticipate errors, such as those handling user input, can be enclosed in a try except block to handle **“exception errors”.**
- The statements to be executed are grouped in a **try :** block and exceptions are passed to the ensuing **except :** block for handling.
- Optionally, this may be followed by a **finally :** block containing statements to be executed after exceptions have been handled.

- Multiple exceptions can be handled by specifying their type as a comma-separated list in parentheses within the except block:

```
except ( NameError , IndexError ) as msg :
    print( msg )
```

```
title = 'Python In Easy Steps'
try :
    print( titel )
except NameError as msg :
    print( msg )
```

Debugging assertions

- When tracking down (debugging) errors in your code it is often useful to “comment-out” one or more lines of code by prefixing each line with the **# hash character** – as used for your comments.
- The Python interpreter will then omit execution of those lines so helps to localize where a problem lies.
- For example, where you suspect a variable assignment problem it can be excluded like this:

```
# elem = elem / 2
```

- If the program now runs without errors the commented-out assignment can be assumed to be problematic.
- Another useful debugging technique employs the Python **assert** keyword to add error checking code to your script.
- This examines a specified test expression for a Boolean **True** or **False** result and reports an “Assertion Error” when the test fails.
- Optionally, an assert statement can include a descriptive message to supply when reporting an Assertion Error, and has this syntax:

  assert *test-expression , descriptive-message*

Assert versus Exception

Important to recognize their distinctions:

- **Exceptions** provide a way to handle errors that may legitimately occur at runtime
- **AssertionErrors** provide a way to alert the programmer to mistakes during development